

# CIS 1990 Final Project: Git Explainer Agent — Design Document

Authors: Andrei and Alistair.

## 1. Problem, user, motivation, and scope

**Problem.** Developers often inherit code whose history is spread across line-level git metadata, commit messages, pull requests, issue threads, review comments, and surrounding source context. Looking this up manually is slow and error-prone, especially when a line has been renamed, moved, refactored, or touched by several commits.

**Intended user.** The primary user is a developer, reviewer, teaching assistant, or maintainer working in a local clone who wants to answer "why does this code exist?" without manually chasing `git blame`, `git log`, GitHub PRs, and issue links.

**Motivation.** We designed the project to make code-history explanations more auditable than a normal chatbot answer. Instead of only producing prose, it returns the retrieved commits, pull requests, issues, file snippets, and diffs that support the explanation, and it requires citations in synthesized claims so users can check the answer.

**Project scope.** The agent takes a local git clone plus either a line range or a natural-language question, and returns a structured JSON `ExplanationResult` with five synthesis sections (`what_changed`, `why`, `tradeoffs`, `limitations`, `summary`); the commits, pull requests, issues, file contexts, and diffs that back them; cache statistics and fallback / condensation audit fields; the `Planner`'s tool-call audit trail; and the `Critic`'s report from an independent second LLM. The Planner and Critic are first-class stages of the agent's pipeline; users turn them on at run time with `--planner` and `--critic`.

## 2. System flow

A single invocation of `python main.py ...` runs the agent through this pipeline:

- **Entry mode:** the CLI accepts either direct line-range input (`repo file_path start end`) or question mode (`repo --question ...` with an optional file hint). `--planner` and `--critic` activate the agent's two LLM-driven stages; `--no-llm` forces the deterministic baseline.
- **Validation:** `validate_query` checks repository scope, file safety, line ranges, public-repo policy, and bounded parameters before any expensive work begins. Invalid input raises `ValueError`.
- **Question resolution:** question mode uses `resolve_question_to_code` to map the question to a concrete file span, then re-runs validation on that resolved span.
- **History tracing:** `trace_line_history` finds commits that shaped the selected lines. If no commits are found, `search_commits` provides a last-resort `git log` fallback.
- **Evidence collection (Planner stage):** with `--planner`, the `Planner LLM` drives an iterative loop that picks the next deterministic tool to invoke from `tool_registry.TOOL_SPECS` — fetching PRs, issues, comments, file context, and diffs — until it returns "done" or hits the iteration cap (`PLANNER_MAX_ITERATIONS`, default 10). When the planner is off (or its LLM is unavailable), the orchestrator runs an equivalent fixed sequence (`_collect_evidence`) so we always produce evidence in the same shape. Both paths share `ExplainerMemory` through `ToolCallContext`, so a planner-driven run reuses cached PRs/issues/diffs from prior fixed-sequence runs and vice versa.
- **Memory layer:** `ExplainerMemory` checks cached PRs, issues, comments, file contexts, and diffs before fetching; every cache miss is written back and flushed at the end.
- **Context control:** `condense_evidence` compresses long PR or issue text only when the evidence payload exceeds `EVIDENCE_CHAR_BUDGET`.
- **Synthesis:** when an LLM is configured, `llm.chat` (Groq llama-3.3-70b by default) writes the cited explanation; otherwise `_fallback_summary` builds deterministic prose. `_ensure_citation_coverage` rejects uncited sentences, retries once, then drops to the fallback if coverage still fails.
- **Critic stage:** with `--critic`, `critic.critique` sends the synthesized draft and a trimmed evidence view to an independent LLM family (Anthropic Claude Haiku 4.5 by default) for a verdict of "ok" or "needs\_more\_evidence". A `needs_more_evidence` verdict drives one re-plan + re-synthesize round through `_run_critic_round` when the planner is also active; otherwise the verdict and its `focus_hints` are recorded as audit data and the draft stands. If the Anthropic SDK or key is missing, the critic returns `verdict="skipped"` so the run still completes.
- **Output:** the CLI prints an `ExplanationResult` JSON object. We return the un-condensed evidence to the caller; only the synthesis prompt sees the condensed copy. The `planner` and `critic` fields carry their respective audit trails when those stages ran, and stay `null` when they did not.

## 3. Design rationale by required component

### Data incorporation

The agent incorporates four main evidence sources: local git history, GitHub pull request metadata, GitHub issue metadata, and source-level context from diffs plus surrounding file snapshots. Local git history is the grounding layer because the user asks about a concrete file span or a question that is resolved into one. GitHub PRs, review comments, issues, and issue comments add intent signals that are often absent from commit messages.

### Memory and retrieval

Retrieval starts with line history because the line span is the most specific signal the user provides. In question mode, the system first uses `question_resolver` to map natural-language text to a concrete file span, then runs the same history pipeline. `ExplainerMemory` caches GitHub and context lookups in `.git_explainer_cache.json` so repeated runs avoid unnecessary API calls and keep evaluation runs more stable.

The cache is intentionally simple and local. We thought that a database or vector index would support richer semantic retrieval, but it would also add setup burden and make the evidence path less transparent. We saw the tradeoff being that the JSON cache can become stale if GitHub conversations change, and it is not designed for concurrent writes by many processes.

### Tools

The system uses small deterministic tools rather than letting an LLM freely choose arbitrary shell commands. Three central examples are `git_blame_trace`, `github_pr_lookup`, and `git_diff_reader`. `git_blame_trace` answers which commits shaped the selected lines; `github_pr_lookup` enriches those commits with review and PR context; `git_diff_reader` extracts compact, redacted change summaries for the synthesis step.

Our main reason for this tool design is control. We found that narrow tools were easier to test, cite, cache, and threat-model than a general command executor. A purely fixed pipeline pays for that control with rigidity — it can miss unusual evidence a human investigator would chase down — so we layered the Planner on top of

the same tools to give the agent adaptive routing while keeping every backend call deterministic and inspectable.

Planner-facing tool registry. The Planner does not call git or GitHub directly. It picks a tool *name* from [TOOL\\_SPECS](#) (an OpenAI-style JSON-schema list) and supplies arguments that match the declared schema; [dispatch\\_tool](#) then runs the deterministic backend with the agent's repo path, owner/repo, and shared cache injected from a side-channel [ToolCallContext](#). The LLM never sees credentials, paths, or remote identifiers because we kept them out of every tool's schema. Adding a new tool is one entry in `TOOL_SPECS` plus one branch in `dispatch_tool`. After each call, [merge\\_tool\\_result](#) folds the return value into a running evidence dict in the same shape the fixed sequence produces, so synthesis is path-agnostic.

Implemented tools. Each tool under [git\\_explainer/tools/](#) is a thin module with one job:

- [git\\_blame\\_trace](#) — primary tracer ([trace\\_line\\_history](#)). `git log -L` first, with `git blame -M` (honoring `.git-blame-ignore-revs`) and `git log --follow -M` filling in commits the line trace missed. Also drops the shallow-clone boundary commit when the parent object is missing locally ([git\\_blame\\_trace.py:184-198](#)), preventing spurious "first introduced here" attributions on partial clones.
- [github\\_pr\\_lookup](#) — `find_prs_for_commit`, `fetch_pr`, `fetch_pr_comments`.
- [github\\_issue\\_lookup](#) — `extract_issue_refs`, `fetch_issue`, `fetch_issue_comments`. Filters out responses where GitHub's `pull_request` field is set, so PRs are not double-counted as issues.
- [file\\_context\\_reader](#) — reads file contents at a given revision.
- [git\\_diff\\_reader](#) — compact per-commit diff summaries with credential redaction ([\\_redact\\_sensitive\\_diff\\_content](#)).
- [commit\\_search](#) — last-resort `git log` wrapper used when line tracing returns nothing.
- [question\\_resolver](#) — maps a natural-language question to a concrete line span using AST parsing for Python files and keyword matching elsewhere. Not an LLM call.

All external fetches are cached in the JSON-backed [ExplainerMemory](#) (stored at `.git_explainer_cache.json` inside the target repo, seven buckets keyed by shape). The tool registry routes both fixed-sequence and planner-driven calls through this same cache.

## Robust system design

We built the agent as a chain of independent fallbacks so that no single upstream failure can take the whole run down. Validation runs before any costly work; `commit_search` covers a line trace that returns no commits; `condense_evidence` shrinks oversized prompts; synthesis runs only when the LLM is available; citation validation rejects uncited prose and retries once; `_fallback_summary` produces deterministic prose when the LLM is disabled or fails; the Planner falls back to the fixed sequence when its LLM is unavailable or hits no successful tool calls ([\\_collect\\_evidence\\_with\\_planner](#)); and the Critic returns `verdict="skipped"` when its provider is missing. The orchestrator records *why* it fell back through the structured `fallback_reason` on [ExplanationResult](#) — `"llm_disabled"`, `"llm_error"` (transient upstream failure), or `"validation_failed"` (synthesis parsed but did not satisfy citation coverage after retry) — so our eval harness and downstream tooling can distinguish a transient upstream outage from a logic-driven fallback.

Our assumption is that a limited, inspectable answer beats failing because one upstream is down, and that unsupported fluent prose is worse than a plain fallback. The limitation is that deterministic fallback summaries are less nuanced, citation coverage only checks for citation-shaped support (not semantic entailment), and the Critic round adds latency and a second API dependency when activated.

## Guardrails

Our guardrails constrain both the user-facing input and the evidence that reaches the model. They validate line ranges, reject missing or binary files, enforce repository containment, cap request sizes, refuse private repositories by default, redact likely credentials from diffs, reject synthesized prose that lacks citations, and bound every Planner-issued tool call against a strict JSON schema.

We wanted the agent to stay useful for normal code-history questions while reducing risk from path traversal, prompt injection, credential exposure, private-repository leakage, and hallucinated answers. The main tradeoff is conservative behavior: some legitimate private or offline workflows require an explicit opt-out (`--allow-private-repo`), and some useful large queries must be narrowed to stay within span and context limits.

We have the following guardrails:

- **Line span** capped at `DEFAULT_MAX_LINE_SPAN = 200` ([guardrails.py:71-76](#)).
- **Positive integers and ordering**: `start_line`, `end_line` > 0 and `end_line` >= `start_line` ([guardrails.py:66-69](#)).
- **File existence**: missing or binary files raise ([guardrails.py:117-125](#)).
- **Repository containment**: [normalize\\_file\\_path](#) rejects paths outside the repo root.
- **Private-repo refusal (default on)**: `enforce_public_repo` now defaults to `True`. The guardrail calls [ensure\\_public\\_github\\_repo](#), which rejects 404 or `private: true`. Opt out at the CLI with `--allow-private-repo` or programmatically by constructing [ExplainerQuery](#) with `enforce_public_repo=False`.
- **Parameter clamping**: `max_commits` stays in `[1, 20]`, and `context_radius` stays in `[0, 200]` ([guardrails.py:102-103](#)).
- **Citation coverage**: [\\_ensure\\_citation\\_coverage](#) rejects synthesized sentences without a bracketed citation and triggers the retry loop.
- **Schema-bounded Planner tool calls**: every entry in [TOOL\\_SPECS](#) declares `additionalProperties: False`, and [\\_validate\\_arguments](#) rejects unknown or wrong-typed fields before the backend runs. The Planner cannot smuggle paths, credentials, or repo identifiers in tool arguments because we kept those in the side-channel [ToolCallContext](#) and out of every schema. Repeated invalid actions abort the loop and hand control back to the deterministic sequence.

## Evaluation

We built evaluation around benchmark cases rather than only manual inspection. The harness checks whether the agent retrieves expected commits, PRs, and issues; whether explanations include citations; whether citations resolve to returned evidence; whether invalid inputs fail safely; and whether latency stays reasonable. We separate retrieval recall from must-abstain precision so that stray PR/issue evidence and missing evidence are scored as distinct failure modes.

We separate retrieval correctness from explanation quality because an answer can retrieve the right commits while still summarizing them poorly, or produce well-formatted citations that do not fully support the prose. Our deterministic faithfulness score is a proxy, so the harness also supports an opt-in [LLM-as-judge](#) scorer that grades each case on a 3-point rubric (`accurate` / `partially accurate` / `hallucinated`) using Anthropic Claude Haiku 4.5 — a different LLM family from our Groq llama-3.3-70b synthesizer, so judge errors are not correlated with synthesis errors. We use the same Anthropic-vs-Groq split at runtime for the Critic. One judge run is one sample and the judge is itself an LLM, so headline numbers should be read alongside the caveats in [eval/FINAL\\_RESULTS.md](#).

## 5. Evidence pre-summarization (condensation)

Long GitHub threads can easily overflow the synthesis model's context window, so between evidence collection and synthesis the orchestrator invokes [condense\\_evidence](#) on the collected payload (commits, pull requests, issues, file contexts, diffs).

Trigger threshold. If the serialized evidence dict is at or under [config.EVIDENCE\\_CHAR\\_BUDGET](#) (default `30000` characters, overridable via the

EVIDENCE\_CHAR\_BUDGET env var), condensation is a no-op and the report's `method_used` is "none". Only when the payload exceeds the budget does the condenser run.

Two-tier strategy. For each eligible field, longest first:

1. Tier 1 (preferred): the LLM is asked for a concise summary ( `EVIDENCE_SUMMARY_TARGET_CHARS`, default 800 ) that explicitly preserves commit SHAs, PR/issue numbers, file paths, technical trade-offs, and stated intent. Output is prefixed with `[pre-summarized]` in the condensed copy.
2. Tier 2 (fallback): deterministic head+tail truncation with a visible elision marker ( `[... content truncated: N chars elided ...]` ). Used when the LLM is unavailable or returns an empty reply. Output is prefixed with `[truncated]`.

Fields touched vs. preserved. Condensation is intentionally narrow:

- **Condensed:** `pull_requests[i].body`, `pull_requests[i].review_comments[j].body`, `issues[i].body`, `issues[i].comments[j].body`, only when length exceeds `config.EVIDENCE_FIELD_MAX_CHARS` (default 3000).
- **Preserved verbatim:** all commit SHAs (full and short), PR/issue numbers, titles, labels, URLs, `file_contexts` entries, `diffs` entries, and any other structural metadata.

Report shape. The condenser returns a `CondensationReport` serialized as the `condensation` field of the `ExplanationResult`:

```
"condensation": {
  "original_size": 48123,
  "condensed_size": 22041,
  "fields_condensed": ["pr#42.body", "issue#7.comments[2].body"],
  "method_used": "llm"    // "none" | "llm" | "heuristic" | "mixed"
}
```

Caller visibility. The `ExplanationResult` we return to the caller still contains the **un-condensed originals** for `pull_requests`, `issues`, `file_contexts`, and `diffs`. Only the synthesis LLM sees the condensed view: `_synthesize` is called with `condensed_evidence` (`orchestrator.py:181`), and inside it `build_synthesis_prompt` feeds that condensed copy to the LLM. The Critic scores against a separately-trimmed body-excerpt view (`_evidence_for_critic`) rather than the full bodies, so its prompt cost stays predictable. Downstream consumers (notebooks, eval harness, `--use-llm-judge`) score the agent against the full evidence, not the compressed view.

## 6. Threat model

The main risks and controls are:

- **C1 – Unsafe line-range input:** invalid spans, non-positive line numbers, oversized ranges, missing files, binary files, and paths outside the repo are blocked by `validate_query` and `normalize_file_path`.
- **C2 – Unsafe question input:** empty, punctuation-only, or stopword-only questions are rejected by `resolve_question_to_code`, which raises when no specific search terms remain.
- **C3 – Prompt injection in user questions:** question resolution is deterministic keyword and path scoring, not an LLM call, so instruction-shaped user text is treated as data.
- **C4 – Prompt injection in fetched evidence:** `SYSTEM_PROMPT` grounds claims in returned evidence; `_ensure_citation_coverage` rejects uncited prose; `_fallback_summary` avoids free-form model text when needed; and the `Critic` runs a second-LLM-family check that flags claims and citations not grounded in the gathered evidence.
- **C5 – Unsafe tool use:** subprocess calls use `argv` lists with `shell=False`, GitHub fetches are scoped to `api.github.com`, and file reads are scoped to the resolved repository root. The Planner cannot call arbitrary commands: it can only emit JSON-shaped tool calls validated against `TOOL_SPECS` and dispatched through `dispatch_tool`, which rejects unknown arguments and wrong-typed values before any backend runs.
- **C6 – Privacy leakage:** `ensure_public_github_repo` refuses private repositories by default; `GITHUB_TOKEN` is only sent to GitHub API requests and is not logged. Repository identifiers (owner, repo, repo path) are kept in the side-channel `ToolCallContext` and never reach the Planner LLM in tool arguments.
- **C7 – Sensitive data in diffs:** `_redact_sensitive_diff_content` masks likely tokens, passwords, API keys, auth headers, and URL credentials before diffs enter the evidence payload.
- **C8 – Hallucinated model output:** citation retry runs up to two attempts, then the orchestrator returns the deterministic fallback instead of unsupported prose. The Critic (independent Anthropic Claude Haiku) layers on a second check that every bracketed citation (e.g. `[pr:#42]`) references an ID that actually appears in the evidence; a `needs_more_evidence` verdict drives one re-plan + re-synthesize round.
- **C9 – Rate-limit and API failures:** GitHub helpers raise clearly on 401, 403, and 429, and retry transient failures with exponential backoff. We catch Planner and Critic LLM failures and record them in their audit reports (`halted_reason="llm_error"` and `verdict="skipped"` with `error` set) so a flaky upstream model never crashes the run.

## 7. Evaluation

We score the agent with `eval/evaluate.py` against 29 benchmark cases in `eval/benchmark.json`. The full snapshot lives in `eval/results.json`, with the dated immutable record in `eval/results_2026-04-25.json` and an annotated summary in `eval/FINAL_RESULTS.md`. Our cases span this project's own history plus external repos (Flask, requests, React, CPython) so retrieval is exercised against real, deep histories we did not author.

Evaluation methodology. We organized the benchmark suite around task types that correspond to the agent's main responsibilities:

- **Line-range explanation tasks:** the user supplies a repository, file, and line span. Success means the agent validates the input, retrieves the expected commits and related PR/issue evidence, and produces a cited explanation for the selected code.
- **Question-resolution tasks:** the user supplies a natural-language question, optionally with a file hint. Success means the resolver maps the question to the expected file/span or matched terms before the normal history-tracing pipeline runs.
- **Thin-evidence and ambiguous-history tasks:** the selected code is connected to multiple commits, weak PR descriptions, or no linked issues. Success means the answer surfaces uncertainty and limitations rather than inventing a single unsupported rationale. The fallback summary explicitly weakens its language ("their descriptions are empty or too brief to document the rationale") when every cited PR/issue body is below 80 characters, so title-only metadata does not get reported as documented intent.
- **LLM/fallback behavior tasks:** some cases run with `use_llm=True` while others force `--no-llm`. Success means the agent either returns an LLM synthesis that passes citation checks or falls back to the deterministic summary with the correct `used_fallback` behavior. The structured `fallback_reason` distinguishes `llm_disabled` from transient `llm_error` so the harness skips (rather than fails) cases blocked by upstream outages.
- **Must-abstain tasks:** spans with no associated PR or issue should produce no PR/issue evidence and no fabricated links. Precision is measured here as zero

- unexpected PRs and zero unexpected issues; recall is not a meaningful target on these cases.
- **Failure cases:** invalid repositories, missing files, binary files, impossible line ranges, and end-line values beyond file length should fail clearly with validation errors instead of running partial tool calls or returning misleading explanations.
  - **Adversarial cases:** prompt-injection-shaped questions, path traversal attempts, and cases that should not return PR/issue metadata test whether the system treats user text as data, keeps file access inside the repo, and avoids unsupported evidence claims.

Primary success criteria: retrieval recall against hand-authored gold commits/PRs/issues, citation coverage, citation validity, must-abstain precision, correct fallback behavior, and correct abstention when PR/issue evidence should be absent. Latency is a secondary operational metric. Failure and adversarial cases are judged by refusal quality, absence of unsafe tool behavior, and whether the agent fabricates evidence. The deterministic faithfulness rubric is a proxy; `--use-llm-judge` adds the cross-provider Claude Haiku rating described above for a stronger signal.

| Metric                                                   | Target | Actual                                                                        |
|----------------------------------------------------------|--------|-------------------------------------------------------------------------------|
| Pass rate                                                | —      | 100% (29 / 29)                                                                |
| Non-trivial pass rate                                    | —      | 100% (28 / 28 cases with ≥1 non-plumbing check)                               |
| Retrieval recall                                         | 85%    | 100% (77 / 77 gold targets across 20 cases)                                   |
| Commit SHA match                                         | —      | 100% (21 / 21)                                                                |
| Must-abstain precision                                   | —      | 100% (11 / 11 cases — 0 stray PRs, 0 stray issues)                            |
| Citation coverage ( <i>format compliance</i> )           | 100%   | 99.0% (201 / 203 citable sentences)                                           |
| Citation validity ( <i>format compliance</i> )           | —      | 100% (340 / 340 citations resolve to real evidence)                           |
| Faithfulness rubric ( <i>proxy, not human-rated</i> )    | 80%    | 3.86 / 5 over 23 scored cases                                                 |
| LLM-judge strict pass rate ( <i>fully accurate</i> )     | —      | 78.3% (18 / 23)                                                               |
| LLM-judge loose pass rate ( <i>accurate or partial</i> ) | —      | 91.3% (21 / 23)                                                               |
| Latency p50                                              | —      | 0.625 s end-to-end                                                            |
| Latency p95                                              | —      | 12.468 s (external-repo cases that fan out into many GitHub PR/issue lookups) |

LLM-judge breakdown: 18 accurate · 3 partially accurate · 2 hallucinated · 0 unscored · 0 skipped. Judge model: `claude-haiku-4-5` (Anthropic). Synthesis model: `llama-3.3-70b-versatile` on Groq. The two remaining `hallucinated` ratings are documented in [eval/FINAL\\_RESULTS.md](#) and are not correctness regressions: one is a judge false positive (the resolver legitimately matched against a transcript file the judge does not see), and one is a run-dependent prose ordering quirk in the synthesis LLM that does not affect retrieval correctness.

## 8. User transcripts

### Transcript 1: Successful line-range query with retrieved evidence

User command.

```
python main.py . git_explainer/guardrails.py 41 60 --no-llm --owner AndreiPiterbarg --repo-name CIS_1990_Final_Project
```

System response excerpt.

```
{
  "commits": [
    {"sha": "b05641e", "message": "adjust to handle natural language query"},
    {"sha": "3870c34", "message": "initial mockup"}
  ],
  "pull_requests": [
    {"number": 1, "title": "initial mockup", "state": "merged"}
  ],
  "explanation": {
    "what_changed": "The selected lines in git_explainer/guardrails.py:41-60 were most recently shaped by 2 traced commit(s): b05641e (initial mockup) and 3870c34 (initial mockup).",
    "why": "Related pull requests suggest the intent was #1 (initial mockup). [pr:#1]",
    "limitations": "This explanation is limited to the traced commits, associated pull requests, linked issues, and any fetched file content."
  },
  "used_fallback": true
}
```

**Behavior shown.** A clean-success path on the deterministic baseline. The agent validates the line range, traces two relevant commits, fetches PR metadata, builds file context and diff evidence, and returns a cited five-section explanation. Because the run uses `--no-llm`, `used_fallback: true` confirms that the deterministic fallback summary produced the final prose, and `fallback_reason: "llm_disabled"` distinguishes this from a transient LLM error. Re-running with `--planner --critic` (and without `--no-llm`) puts the agent into its full pipeline: the `planner` field then carries the LLM-driven tool-call audit trail — each entry records the chosen tool, arguments, status, and a one-line result summary — and the `critic` field carries a verdict of "ok" / "needs\_more\_evidence" / "skipped" from the independent Anthropic Claude Haiku critic. We force both flags off in `main.py` whenever `--no-llm` is set, so the `planner` and `critic` fields stay null here.

## Transcript 2: Difficult case with multi-commit, ambiguous history

User command.

```
python main.py . git_explainer/config.py 13 19 --no-llm --owner AndreiPiterbarg --repo-name CIS_1990_Final_Project
```

System response excerpt.

```
{
  "commits": [
    {"sha": "3870c34", "author": "aking526", "message": "initial mockup"},
    {"sha": "4c711bf", "author": "AndreiPiterbarg", "message": "Switch LLM provider from Kimi to Groq"},
    {"sha": "628ef3c", "author": "AndreiPiterbarg", "message": "Add project configuration and dependencies"}
  ],
  "explanation": {
    "what_changed": "The selected lines in git_explainer/config.py:13-19 were most recently shaped by 3 traced commit(s): 3870c34 (initial mockup), 4c711bf (Switch LLM provider from Kimi to Groq), and 628ef3c (Add project configuration and dependencies).",
    "why": "Related pull requests suggest the intent was #1 (initial mockup) [pr:#1].",
    "limitations": "This explanation is limited to the traced commits, associated pull requests, linked issues, and any fetched file content."
  },
  "used_fallback": true
}
```

**Behavior shown.** The chosen config lines were touched by multiple commits from different authors: initial configuration, provider migration, and later refactoring. The agent does not collapse that evidence into a single unsupported narrative. It cites all three commits, includes the related PR when available, and uses the `limitations` field to flag that intent may be incomplete when discussion is absent from retrieved metadata. We captured this transcript before adding `_is_substantive_artifact`; on the current code this same query produces the softer "their descriptions are empty or too brief to document the rationale" lead in the `why` field, because PR #1's body (50 chars) is below our 80-char threshold and saying "the intent was *initial mockup*" over-claims from a title alone.

## Transcript 3: Safety case — adversarial low-signal question rejected by C2

User command.

```
python main.py . --question "explain how this is used" --owner AndreiPiterbarg --repo-name CIS_1990_Final_Project --no-llm
```

System response (stderr, exit code 1).

```
Traceback (most recent call last):
  File "main.py", line 107, in <module>
    main()
  File "main.py", line 88, in main
    result = explain_code_history(
  File "git_explainer/orchestrator.py", line 876, in explain_code_history
    return agent.explain(query)
  File "git_explainer/orchestrator.py", line 110, in explain
    resolution = resolve_question_to_code(
  File "git_explainer/tools/question_resolver.py", line 139, in resolve_question_to_code
    raise ValueError("question must include at least one specific search term")
ValueError: question must include at least one specific search term
```

**Behavior shown.** The question is an imperative-styled probe ("explain how this is used") whose every token — `explain`, `how`, `this`, `is`, `used` — is in the resolver's stopword set ([question\\_resolver.py:38-85](#)). After stopword filtering the term list is empty, so [\\_extract\\_question\\_features](#) returns no terms, and [resolve\\_question\\_to\\_code](#) raises at line 139 — our C2 control in the threat model. The agent never reaches `trace_line_history`, never calls the GitHub API, and never invokes the synthesis LLM, so a vacuous adversarial prompt cannot trick the model into fabricating an answer or enumerating repository contents. The same guard fires for empty questions and pure-punctuation questions because both reduce to an empty term list.